

Consulta SQL Avançado

Desconstruindo a Caixa Preta do Banco de Dados



Professor

Gustavo Dias
Mestre em Gestão e Desenvolvimento Regional

Consultor em Business Intelligence, com atuação em projetos na área da saúde e varejo.

Contatos:

 p.gustavodias@gmail.com

 <https://www.linkedin.com/in/pgustavodias/>

 pgustavodias





Introdução

Considere as consultas SQL abaixo.
Qual delas será processada mais rapidamente?

```
-- SQL
```

```
SELECT * FROM cliente WHERE cpf = '12345678900';
```

```
SELECT * FROM cliente WHERE nome = 'Maria';
```



Introdução

Diferentes fatores podem refletir para a performance do processamento de uma consulta em um banco de dados.

Podemos categorizá-los como:

- Estrutura da tabela;
- Índices;
- Volume de dados;
- Consulta;
- Concorrência;
- Hardware.

A Sintaxe Correta

```
SELECT * FROM Users  
WHERE user_id = 'A123'  
AND active = TRUE;  
  
UPDATE Inventory  
SET stock = stock - 1  
WHERE product_id = 'P789'  
AND location = 'Warehouse-A';  
  
INSERT INTO Logs  
  (event_type, timestamp, user)  
VALUES ('LOGIN', NOW(), 'A123');
```

O programador especifica o que deseja.

A Execução Eficiente



O SGBD decide como obter o resultado. Em bancos com milhões de registros, a diferença de execução pode ser de milissegundos para vários minutos.



Como o Otimizador Escolhe a Rota?

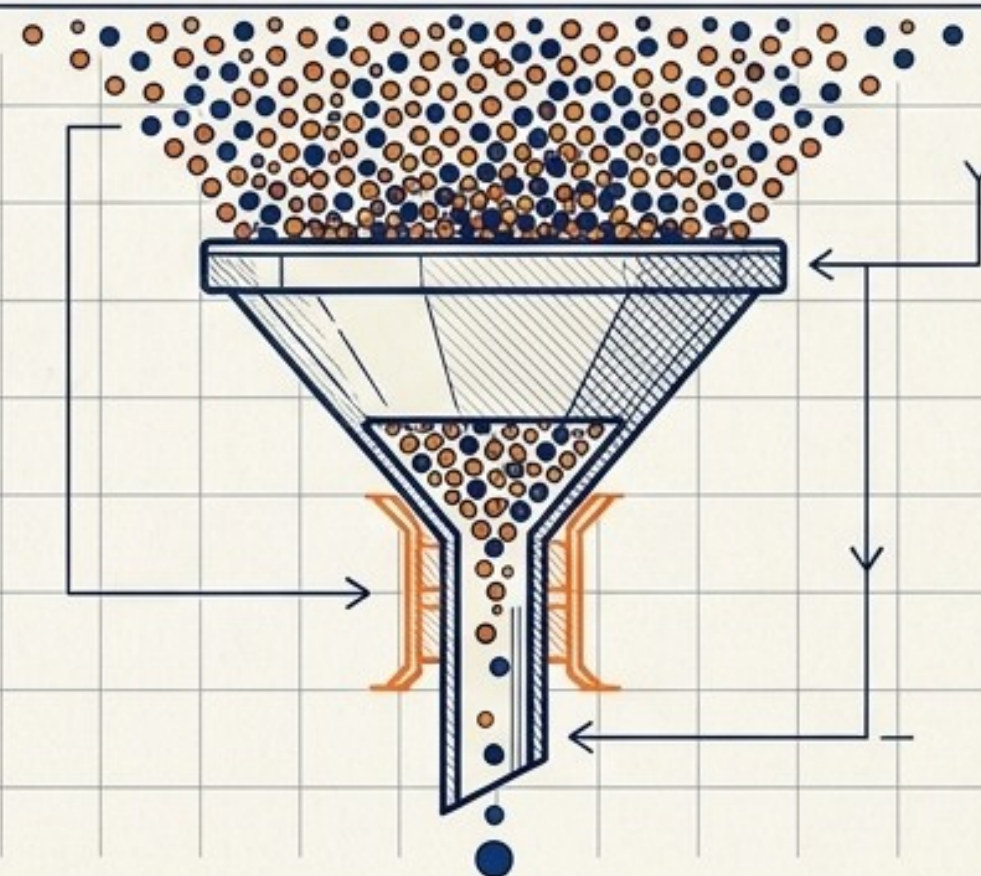


Cardinalidade

Número de valores distintos em uma coluna.

Exemplo (1.000.000 de registros):

- CPF = **Alta** Cardinalidade (**Eficiente**)
- Sexo = **Baixa** Cardinalidade (**Ineficiente**)

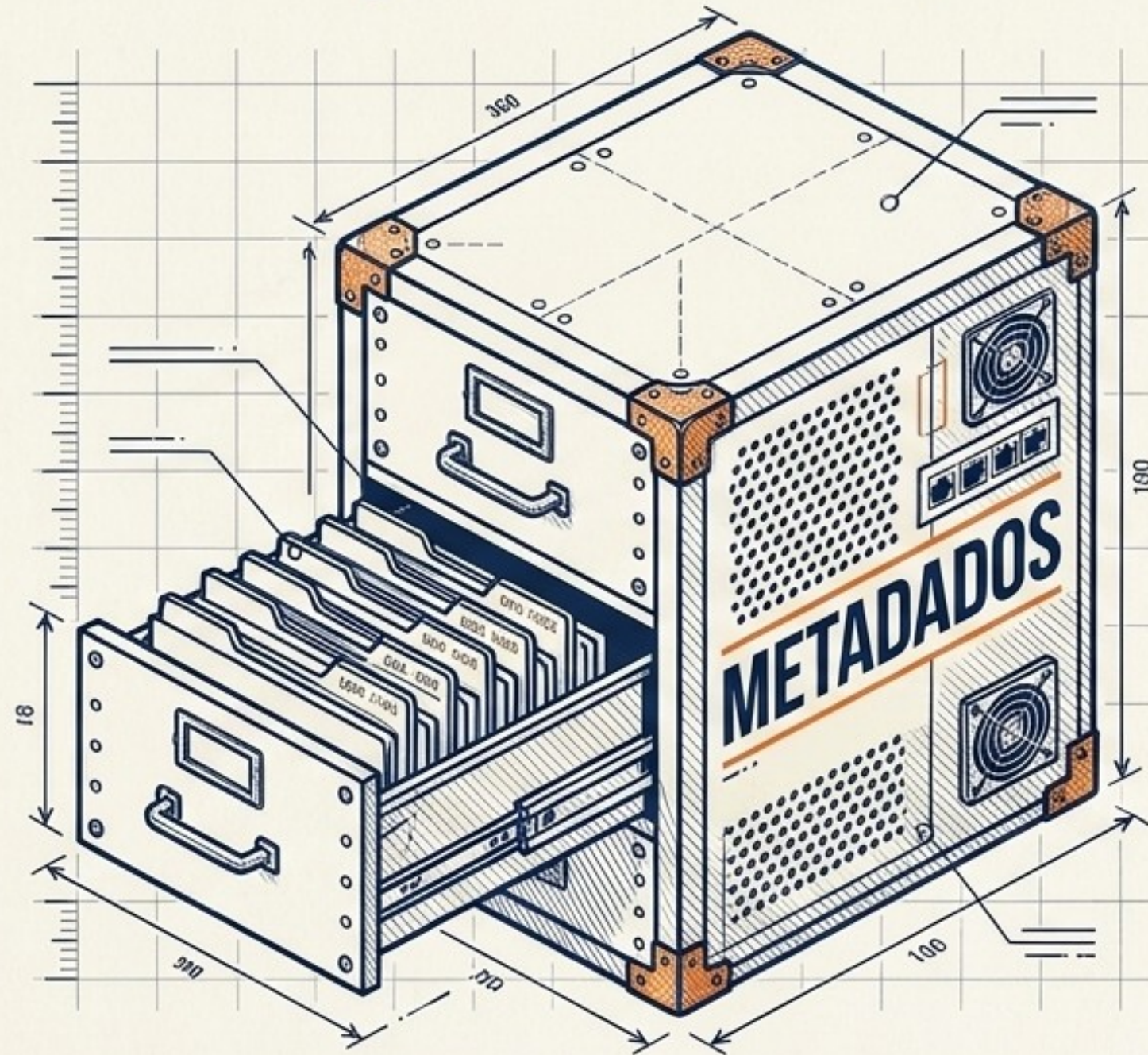


Seletividade

Capacidade de um índice reduzir o conjunto de registros analisados.

Quanto **maior** a **seletividade**, maior a probabilidade de o índice ser utilizado.

O Catálogo de Dados (Data Dictionary)



O SGBD mantém informações sobre si mesmo: Tabelas, Colunas, Índices, Restrições, Permissões, Estatísticas.

```
SELECT table_name, engine, table_rows  
FROM information_schema.tables  
WHERE table_schema = 'meubanco';
```

O dicionário de dados é um componente essencial consumido pelo próprio otimizador de consultas (Elmasri e Navathe, 2017).

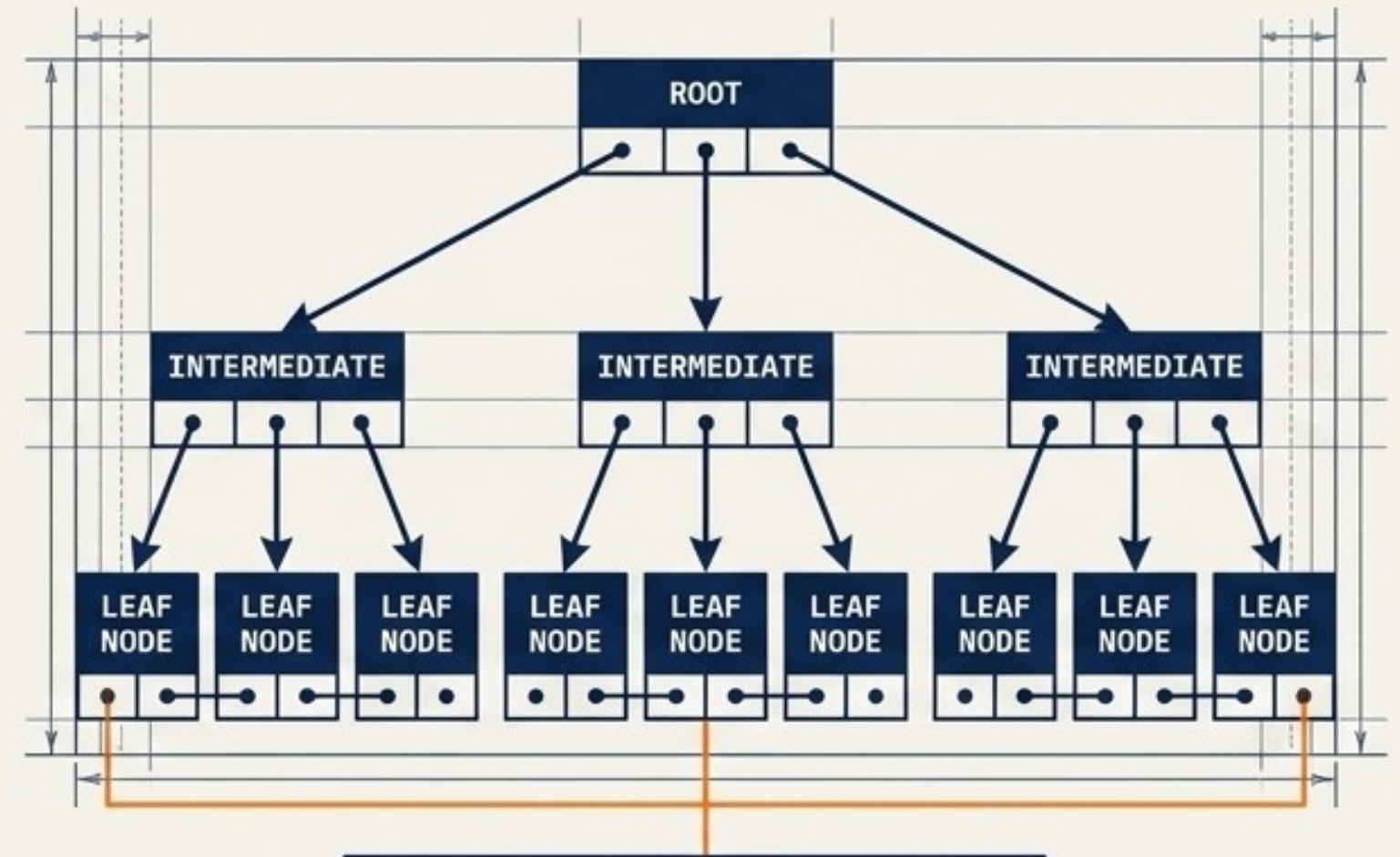
Índices: O Atalho do Motor



A Metáfora

Funciona como o índice de um livro. Evita a leitura de todas as páginas para encontrar um registro específico.

Responsáveis pelos maiores ganhos de desempenho (Schwartz, Zaitsev e Tkachenko, 2012).



A Realidade Técnica

O motor InnoDB do MySQL utiliza B-Trees.

- Busca em tempo logarítmico
- Boa performance para igualdade e intervalos

Estruturas de Índices e a Regra do Prefixo

Índice Primário: Associado à chave primária.

Índice Secundário: Criado sobre outras colunas.

Índices Compostos: Índice criado em (cidade, idade)

Cenário A: WHERE cidade = 'Poços de Caldas'



Índice Utilizado

Cenário B: WHERE cidade = 'Poços' AND idade = 30



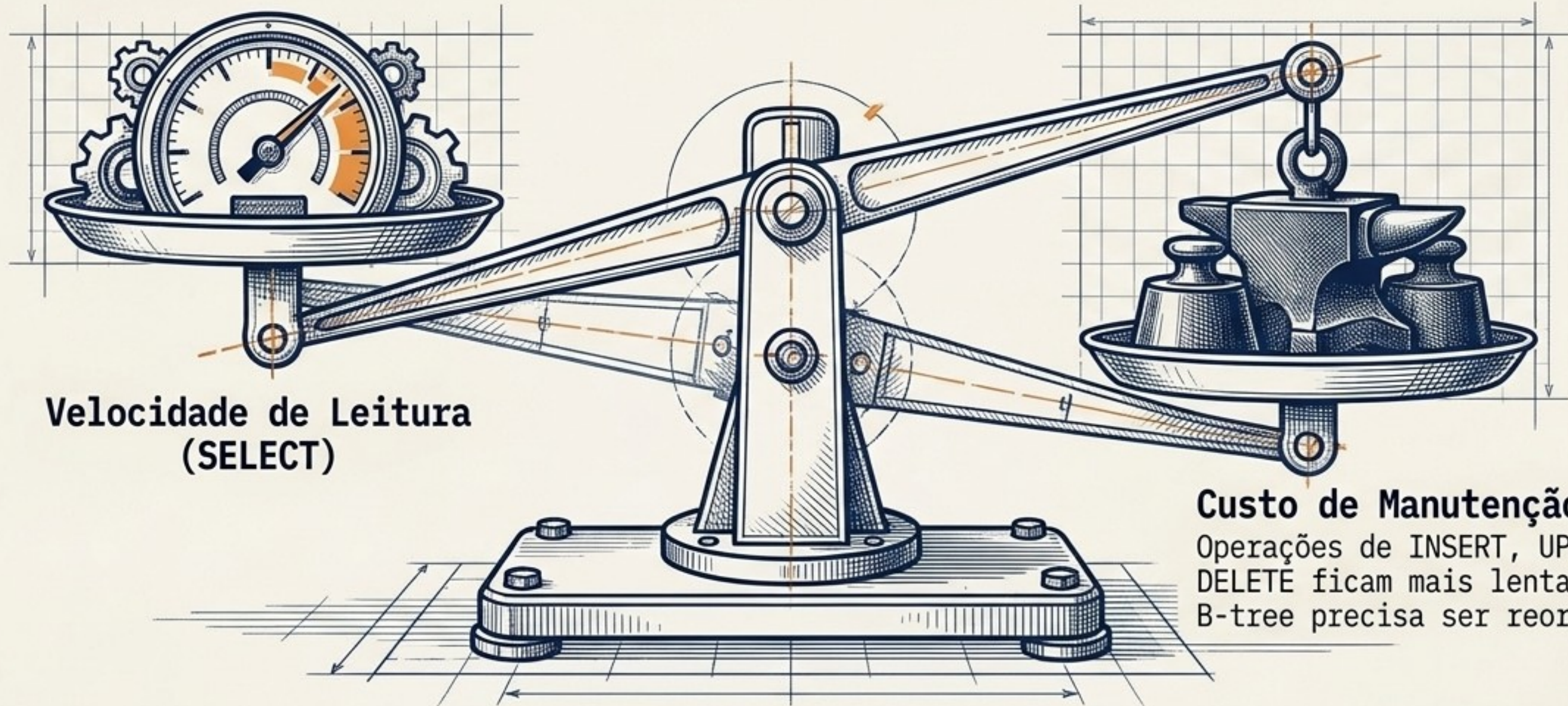
Índice Utilizado

Cenário C: WHERE idade = 30



Índice Ignorado
(Não segue a regra do prefixo à esquerda)

A Balança de Trade-offs: Quando o Índice Pesa



**Velocidade de Leitura
(SELECT)**

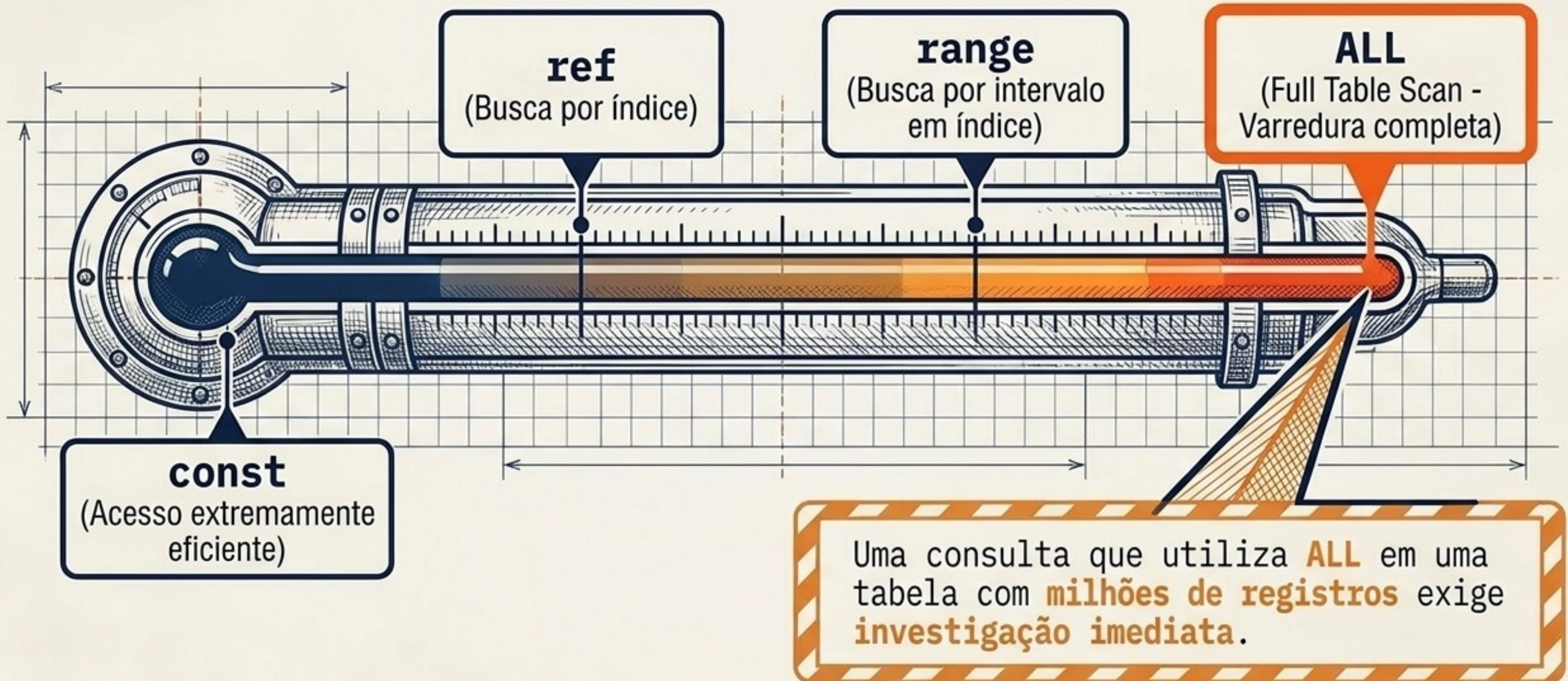
Custo de Manutenção

Operações de INSERT, UPDATE e DELETE ficam mais lentas pois a B-tree precisa ser reorganizada.

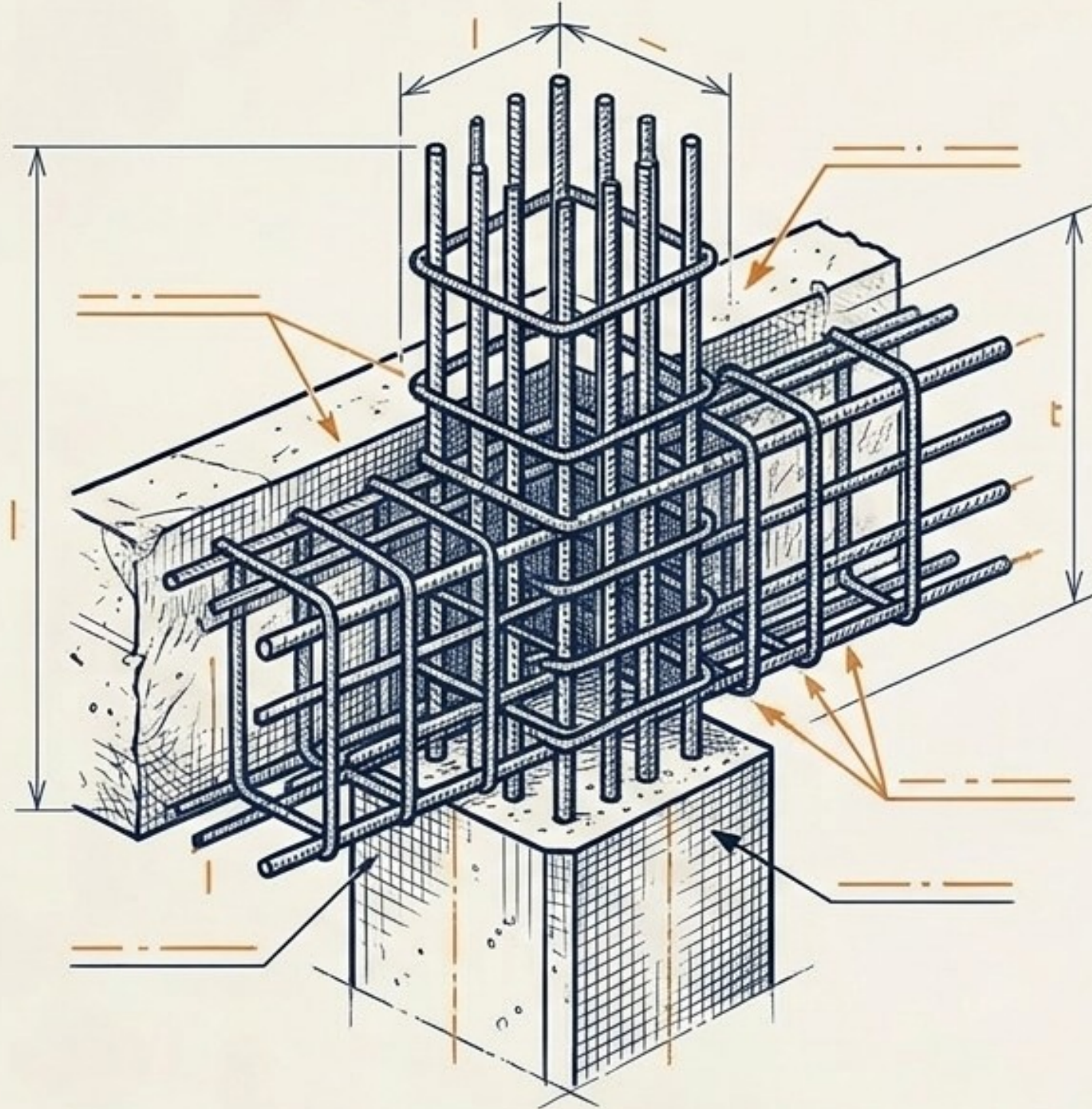
Índices não ajudam quando: colunas têm poucos valores distintos (ex: ativo/inativo), a tabela é muito pequena, ou a consulta retorna quase todos os registros.

EXPLAIN: O Painel de Diagnóstico

O comando **EXPLAIN** antes da consulta revela o plano de execução do MySQL.



Restrições (Constraints) Ajudam o Otimizador



PRIMARY KEY & UNIQUE

Não apenas garantem consistência, mas geram automaticamente índices no motor InnoDB.

FOREIGN KEY, NOT NULL, CHECK

Fornecem regras rígidas. O banco utiliza essas garantias para eliminar possibilidades e atalhar caminhos durante a execução da consulta, sem precisar ler os dados fisicamente.

Views: Abstração vs. Desempenho

Expectativa

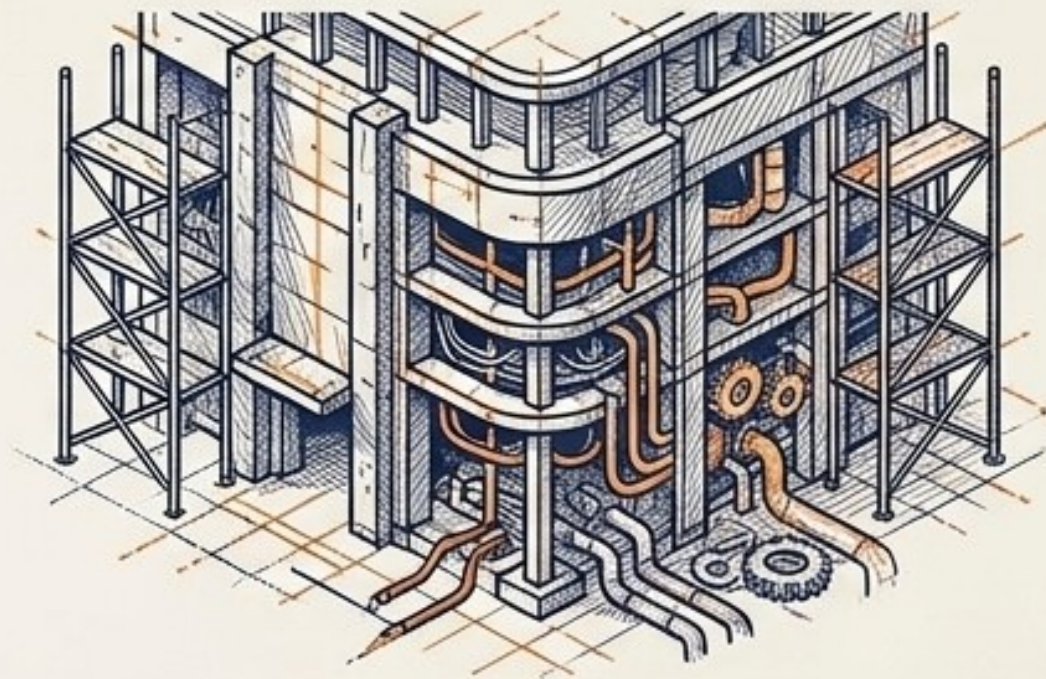


Views melhoram o desempenho por serem consultas armazenadas.

Vantagens Reais:

- Simplificação de consultas complexas.
- Reutilização de lógica.
- Controle de acesso.

Realidade (MySQL)



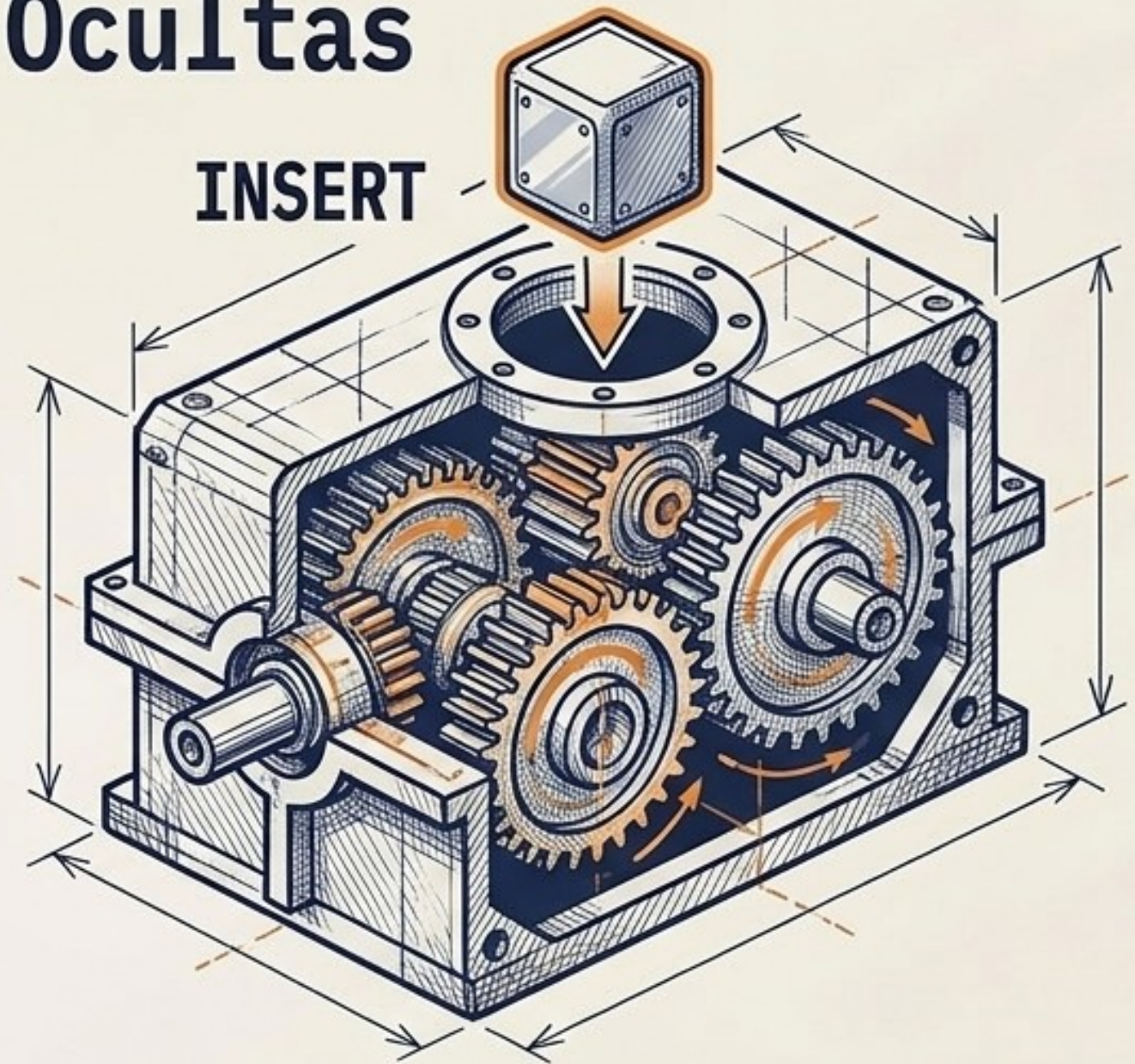
Em geral, não. A view é expandida durante o processamento e tratada como a consulta original.

Nota: Outros SGBDs possuem Materialized Views (armazenamento físico) para ganhos analíticos reais.

Triggers: As Engrenagens Ocultas

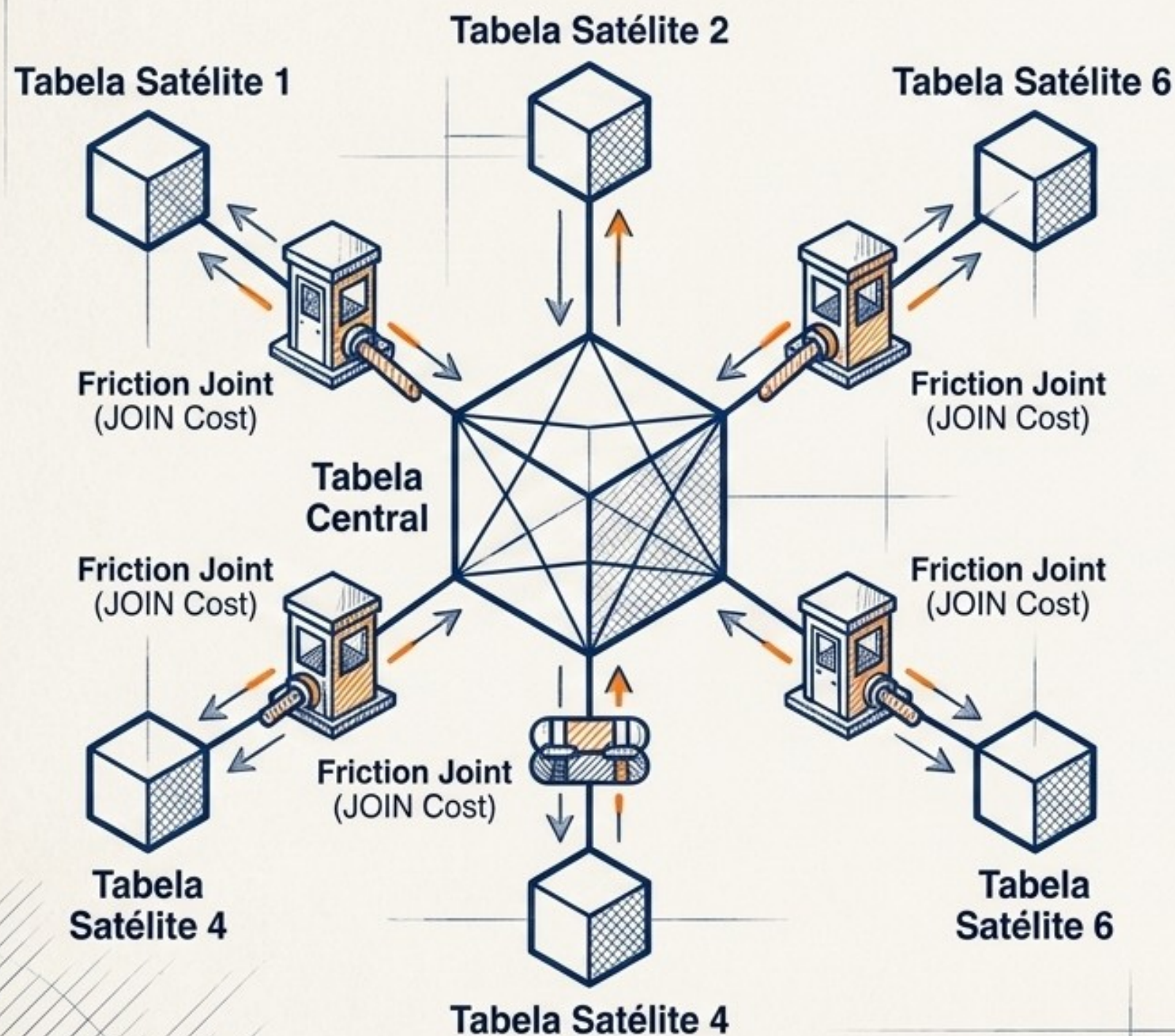
Executados automaticamente em eventos de INSERT, UPDATE ou DELETE.

Aplicações: Auditoria, histórico, validações, regras de negócio.



Impacto de Performance: Adicionam processamento extra às operações de escrita. Um simples INSERT pode disparar múltiplas operações, aumentando o tempo da transação de forma invisível ao usuário final.

O Custo da Normalização



A normalização reduz **redundância** e garante consistência.

Porém, modelos altamente normalizados exigem múltiplos **JOINS**.

The Trade-off

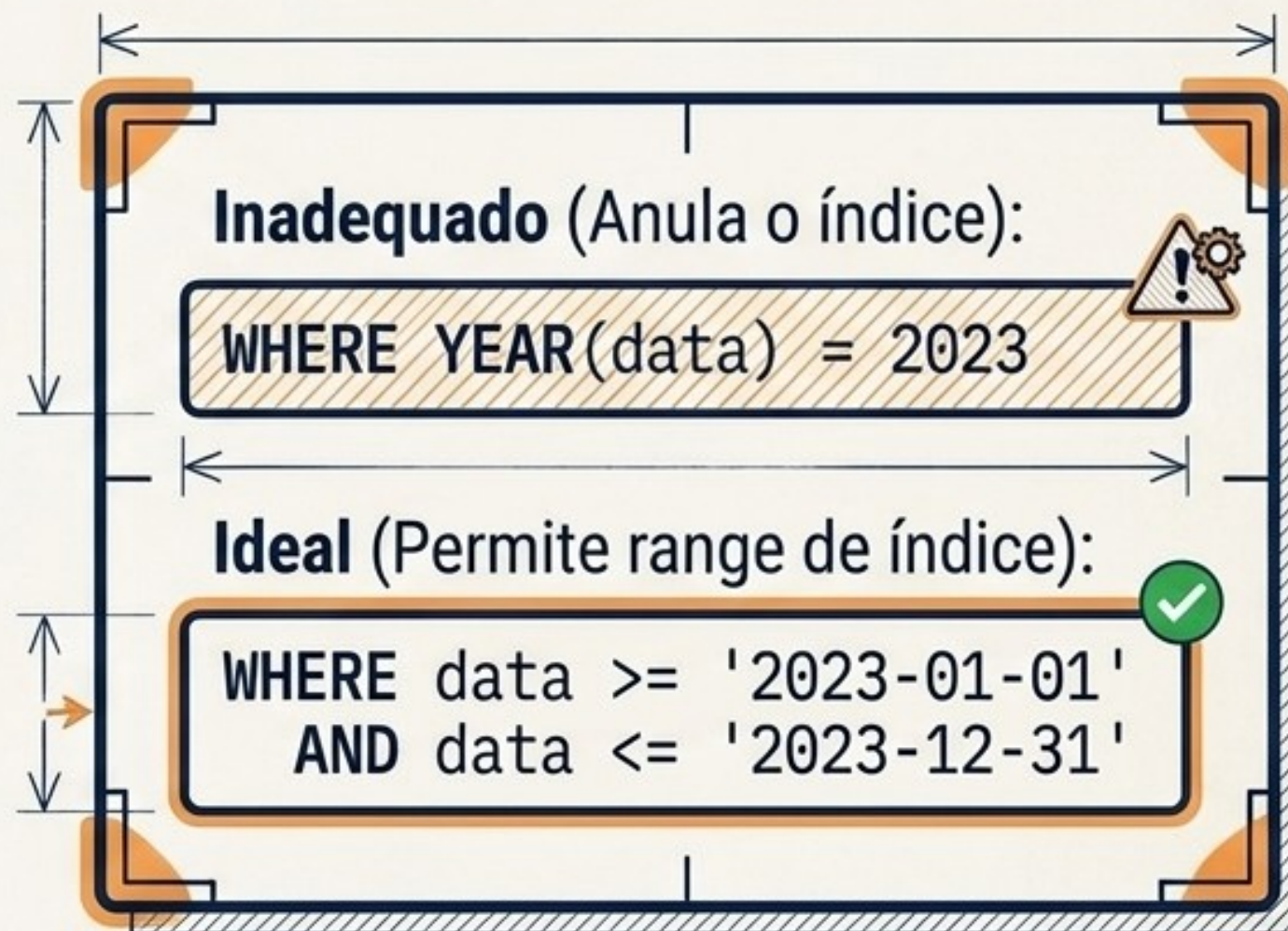
- Cada **JOIN** representa **trabalho adicional** para o **motor** do banco.
- Em sistemas **analíticos** (Data Warehouse), utiliza-se **desnormalização controlada** para reduzir junções e **favorecer a leitura rápida** (Kimball e Ross, 2013).

Checklist do Desenvolvedor de Alta Performance

Lista de Verificação

- ☐ Existe **índice** nas colunas do WHERE?
- ☐ Os JOINS utilizam **colunas indexadas**?
- ☐ A consulta retorna **apenas as colunas necessárias**? (Evitar SELECT *)
- ☐ O EXPLAIN indica uso do índice (Sem ALL)?

A Armadilha das Funções



O desempenho depende mais de entender os bastidores do SGBD do que da sintaxe da linguagem.

Obrigado!

